C07 Sprite Animation

CRE132 worksheet

What you'll make

A player who breathes while standing still, walks when you move, and turns to face the way they're going, plus slimes that bob on the spot. The movement is C04's **SimplePlayerMovement**; this sample adds the animation.

You'll make **animation clips** from sprite sheet frames, build an **Animator Controller** with two states, a **parameter** and **transitions**, and write a short script, **PlayerAnimator**, that tells the Animator how fast the player is going.



Figure 1: The finished scene: the player in a grass field with three slimes and some flowers.

Before you start

1. Unzip `CRE132_C07_SpriteAnimation_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_C07_SpriteAnimation_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Core/C07_SpriteAnimation`.
3. Press **Play**. The three **Slimes** sit still. The **Player** walks with WASD or the arrows (C04's script), but slides around frozen on one frame, always facing right. Press Play again to stop.

The Player and the Slimes each have an **Animator** component, but its **Controller** slot is empty: there are no clips or controllers yet. You'll make them, then write the script. Your clips go in `Assets/_game/Animation/Clips/MysticWoods` and your controllers in `Assets/_game/Animation/Controllers`, the same places as in the Finished project. If a folder is missing, make it (right-click in the Project window, **Create > Folder**).

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. A clip the quick way: a slime

1. In `Assets/_game/Art/Sprites/MysticWoods`, click the arrow on `slime.png`. Click `slime_0`, Shift-click `slime_3` to select all four, and drag them into the Scene view.
2. Unity asks where to save an animation clip. Go to `Assets/_game/Animation/Clips/MysticWoods` and call it `Slime_Idle`.
3. Unity has done three things at once: made the **clip**, made an **Animator Controller** next to it, and added a new slime to the scene with an **Animator** component that runs that controller.
4. The new slime is on the **Default** sorting layer, so it's hidden under the grass. Select it in the Hierarchy and set its Sprite Renderer's **Sorting Layer** to **Characters**, like the other slimes.

Test it: press Play. The new slime bobs; the three old ones still sit still.

Now give the old slimes the same animation:

1. In the Project window, find the new controller next to `Slime_Idle`. Drag it into `Assets/_game/Animation/Controllers` and rename it `Slime` (select it and press F2).
2. In the Hierarchy, click the first **Slime** and Ctrl-click the other two, so all three are selected. Drag the `Slime` controller into the **Controller** slot of their **Animator**.
3. Delete the slime you dragged in (select it and press Delete). You only needed it to make the clip.

Test it: press Play. All three slimes bob.

2. The Animation window

Open **Window > Animation > Animation** and dock it at the bottom. Select a **Slime**: the window shows `Slime_Idle`. It has one track, **Sprite**, with a **keyframe** (a diamond) for each frame: at each one, the Sprite Renderer switches to the next sprite.

The frame rate is called **Samples**. If you can't see it, open the window's menu (the three dots at the top right) and choose **Show Sample Rate**. Set it to **6**, the speed of the Finished slime. Try 12 and 3 and press Play each time to see the slimes speed up and slow down, then set it back to **6**.

The player's clips you make by hand:

1. Select the **Player**. The Animation window says there is nothing to animate yet and shows a **Create** button. Click it, go to `Assets/_game/Animation/Clips/MysticWoods` and save the clip as `Player_Idle`.
2. Set **Samples** to **8**.
3. In the Project window, open `player.png`. Click `player_6` and Shift-click `player_11` (the second row of the sheet: idle, facing right). Drag the six sprites onto the timeline on the right of the Animation window. A **Sprite** track appears with a keyframe for each one.
4. Click the clip drop-down at the top left of the Animation window (it says `Player_Idle`) and choose **Create New Clip...**. Save it as `Player_Walk` in the same folder.
5. Set **Samples** to **10**, then drag `player_24` to `player_29` (walking, facing right) onto the timeline.

When you made `Player_Idle`, Unity also made an Animator Controller called `Player` next to it and put it in the Player's Animator. Drag `Player` into `Assets/_game/Animation/Controllers`, next to `Slime`.

Test it: press Play. The player breathes while standing still, but walking still shows the idle.

3. The Animator Controller

A clip only says *what* to show. An **Animator Controller** decides *which* clip plays, and when.

1. Select the **Player** and open **Window > Animation > Animator**. It shows the `Player` controller: **Entry**, **Any State**, **Exit** and two **states**, one for each clip you made.
2. **Player_Idle** is orange: it's the **default state**, the one that plays first. **Player_Walk** is grey.
3. Rename the states **Idle** and **Walk** (select one, then change the name at the top of the Inspector).

A parameter: on the **Parameters** tab at the left of the Animator window, click **+**, choose **Float** and name it `Speed`. Code will set this number; the controller reads it.

Transitions:

1. Right-click **Idle**, choose **Make Transition** and click **Walk**. An arrow appears. Click the arrow and in the Inspector: untick **Has Exit Time**, open **Settings** and set **Transition Duration** to **0**. Under **Conditions**, click **+** and set it to **Speed**, **Greater**, **0.1**.
2. Make a transition from **Walk** back to **Idle** the same way, with the condition **Speed**, **Less**, **0.1**.

- **Has Exit Time off** means the switch happens as soon as the condition is true, not when the clip ends.
- **Transition Duration 0** switches straight away. Pixel art can't blend between two frames, so a blend just feels slow.

Test it: press Play. The player breathes, but still never walks: nothing sets **Speed** yet.

4. Telling the Animator: PlayerAnimator

Make a folder `Assets/_game/Scripts/C07_SpriteAnimation` and create `PlayerAnimator` in it:

```
// Tells the Animator how fast the player is moving, so it can switch between Idle and
// Walk,
// and flips the sprite so the player faces the way it is walking.
// It only watches the player's position, so it works with any movement script (here
// C04's SimplePlayerMovement).
using UnityEngine;

public class PlayerAnimator : MonoBehaviour
{
    private Animator animator;
    private SpriteRenderer spriteRenderer;
    private Vector3 lastPosition;

    private void Awake()
    {
        animator = GetComponent<Animator>();
        spriteRenderer = GetComponent<SpriteRenderer>();
        lastPosition = transform.position;
    }

    private void LateUpdate()
    {
        // LateUpdate runs after every Update, so the movement script has already
        // moved us this frame.
        // Speed = distance moved since last frame / time since last frame.
        float distance = Vector3.Distance(transform.position, lastPosition);
        float speed = distance / Time.deltaTime;

        // "Speed" must match the parameter name in the Animator Controller exactly.
        animator.SetFloat("Speed", speed);

        // The sprites face right. Flip them when moving left; keep the last direction
        // when moving straight up or down.
```

```
        if (transform.position.x < lastPosition.x)
        {
            spriteRenderer.flipX = true;
        }
        else if (transform.position.x > lastPosition.x)
        {
            spriteRenderer.flipX = false;
        }

        lastPosition = transform.position;
    }
}
```

What's going on:

- **Speed is worked out from the position**: how far the player moved since last frame, divided by the time that took. The script doesn't need to know which movement script moved the player, so it works with any of them.
- **`LateUpdate`** runs after every object's `Update`, so SimplePlayerMovement has already moved the player this frame.
- **`animator.SetFloat("Speed", speed)`** is how code talks to the Animator: through parameters. The name in quotes must match the parameter exactly, capital S included.
- **`flipX`** mirrors the sprite. The art faces right, so it's flipped when moving left. Moving straight up or down changes neither, so the player keeps facing the last way.

Add it: select the **Player** and **Add Component > PlayerAnimator**. There are no slots to fill.

5. Watch it live

Put the **Animator** window next to the Game view, select the **Player** and press Play. The playing state shows a blue progress bar, and the **Speed** parameter changes as you walk.

Test it: walk left and right. The walk plays and the player faces the way they're going. Stop and they go back to idle straight away.

Check your work

- ☐ All three slimes bob, using `Slime_Idle` (6 samples) and the `Slime` controller.
- ☐ `Player_Idle` (`player_6` to `player_11`, 8 samples) and `Player_Walk` (`player_24` to `player_29`, 10 samples) are in `Animation/Clips/MysticWoods`.
- ☐ The Player's Animator uses `Player`, with Idle as the default state.
- ☐ Walking plays the walk; stopping plays idle, with no delay.
- ☐ The player faces left when walking left and keeps facing that way when walking up or down.
- ☐ Compare with `CRE132_C07_SpriteAnimation_Finished` if something behaves differently.

Key points

- **A clip** is a list of keyframes over time (here, which sprite to show). **An Animator Controller** chooses which clip plays. **The Animator** component on the object runs the controller.
- **Code talks to the Animator only through parameters** (`SetFloat`, `SetBool`, `SetTrigger`). The names must match exactly.
- **For pixel art:** no exit time and zero transition duration, or the switch feels slow.
- **`LateUpdate`** runs after all the `Update`s, so the player has already moved this frame.

If something goes wrong

- **“Parameter ‘Speed’ does not exist” in the Console:** the parameter in your controller is spelt differently (`speed`, `Speed` with a space). Rename it.
- **The walk starts late or finishes its loop before stopping: Has Exit Time** is still ticked on a transition, or the Transition Duration isn’t 0.
- **The player never walks:** the transition conditions are the wrong way round (Idle to Walk should be **Greater**), or PlayerAnimator isn’t on the Player.
- **NullReferenceException in PlayerAnimator:** the Player has no **Animator** component.
- **The player slides around frozen on one frame:** the Animator’s **Controller** slot is empty.
- **A slime I dragged in is invisible:** it’s on the **Default** sorting layer, under the grass. Set it to **Characters**.
- **A clip plays far too fast:** its **Samples** is still 60 or 12. Set it to the number in the step.
- **The Animation window shows the wrong clip:** pick the right one in the clip drop-down at its top left, or select the right object in the Hierarchy.

Try it

1. Add a **Run** state that plays `Player_Walk` faster (set the state’s **Speed** to 1.5 in the Inspector) when Speed is greater than 6 (hold Shift to run).
2. Walk straight up or down: the player still shows the side walk. T02 uses a blend tree for four directions.
3. Make a slime play its hop instead: `slime_21` to `slime_26` (row 4 of the sheet).